In [ ]:
```
Online Shopping Customer Purchase Prediction

Objective:
To predict whether an online visitor will make a purchase during a browsing
session using machine learning classification techniques.
```

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

from sklearn.inspection import permutation_importance

import warnings
warnings.filterwarnings("ignore")
```

In [2]:
```
df = pd.read_csv("online_shoppers_intention.csv")

print("Dataset loaded successfully!")
print("Shape:", df.shape)

df.head()
```

```
Dataset loaded successfully!
Shape: (12330, 18)
```

Out[2]:

	Administrative	Administrative_Duration	Informational	Informational_Duration	Prod
0	0	0.0	0	0.0	
1	0	0.0	0	0.0	
2	0	0.0	0	0.0	
3	0	0.0	0	0.0	
4	0	0.0	0	0.0	

In [3]:
```
print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])
```

```
Number of rows: 12330
Number of columns: 18
```

In [4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12330 entries, 0 to 12329
Data columns (total 18 columns):
 #   Column                   Non-Null Count  Dtype  
---  ------                   --------------  -----  
 0   Administrative           12330 non-null  int64  
 1   Administrative_Duration  12330 non-null  float64
 2   Informational            12330 non-null  int64  
 3   Informational_Duration   12330 non-null  float64
 4   ProductRelated           12330 non-null  int64  
 5   ProductRelated_Duration  12330 non-null  float64
 6   BounceRates              12330 non-null  float64
 7   ExitRates                12330 non-null  float64
 8   PageValues               12330 non-null  float64
 9   SpecialDay               12330 non-null  float64
 10  Month                    12330 non-null  object 
 11  OperatingSystems         12330 non-null  int64  
 12  Browser                  12330 non-null  int64  
 13  Region                   12330 non-null  int64  
 14  TrafficType              12330 non-null  int64  
 15  VisitorType              12330 non-null  object 
 16  Weekend                  12330 non-null  bool   
 17  Revenue                  12330 non-null  bool   
dtypes: bool(2), float64(7), int64(7), object(2)
memory usage: 1.5+ MB
```

In [5]:
```
df.describe(include="all")
```

Out[5]:

	Administrative	Administrative_Duration	Informational	Informational_Duration
count	12330.000000	12330.000000	12330.000000	12330.000000
unique	NaN	NaN	NaN	NaN
top	NaN	NaN	NaN	NaN
freq	NaN	NaN	NaN	NaN
mean	2.315166	80.818611	0.503569	34.472398
std	3.321784	176.779107	1.270156	140.749294
min	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000	0.000000
50%	1.000000	7.500000	0.000000	0.000000
75%	4.000000	93.256250	0.000000	0.000000
max	27.000000	3398.750000	24.000000	2549.375000

In [6]:
```
df.columns
```

Out[6]:
```
Index(['Administrative', 'Administrative_Duration', 'Informational',
       'Informational_Duration', 'ProductRelated', 'ProductRelated_Duration',
       'BounceRates', 'ExitRates', 'PageValues', 'SpecialDay', 'Month',
       'OperatingSystems', 'Browser', 'Region', 'TrafficType', 'VisitorType',
       'Weekend', 'Revenue'],
      dtype='object')
```

In [7]:
```
df.dtypes
```

Out[7]:
```
Administrative             int64
Administrative_Duration    float64
Informational              int64
Informational_Duration     float64
ProductRelated             int64
ProductRelated_Duration    float64
BounceRates                float64
ExitRates                  float64
PageValues                 float64
SpecialDay                 float64
Month                      object
OperatingSystems           int64
Browser                    int64
Region                     int64
TrafficType                int64
VisitorType                object
Weekend                    bool
Revenue                    bool
dtype: object
```

In [8]:
```
print("Missing values:")
print(df.isnull().sum())
```

```
Missing values:
Administrative             0
Administrative_Duration    0
Informational              0
Informational_Duration     0
ProductRelated             0
ProductRelated_Duration    0
BounceRates                0
ExitRates                  0
PageValues                 0
SpecialDay                 0
Month                      0
OperatingSystems           0
Browser                    0
Region                     0
TrafficType                0
VisitorType                0
Weekend                    0
Revenue                    0
dtype: int64
```

In [9]:
```
print("Total missing values:", df.isnull().sum().sum())
```

```
Total missing values: 0
```

In [10]:
```
print("Number of duplicate rows:", df.duplicated().sum())
```

```
Number of duplicate rows: 125
```

In [11]:
```
df = df.drop_duplicates()

print("Shape after removing duplicates:", df.shape)
```

```
Shape after removing duplicates: (12205, 18)
```

In [12]:
```
print(df["Revenue"].value_counts())
```

```
Revenue
False    10297
True      1908
Name: count, dtype: int64
```

In [13]:
```
print(df["Revenue"].value_counts(normalize=True) * 100)
```

```
Revenue
False    84.367063
True     15.632937
Name: proportion, dtype: float64
```

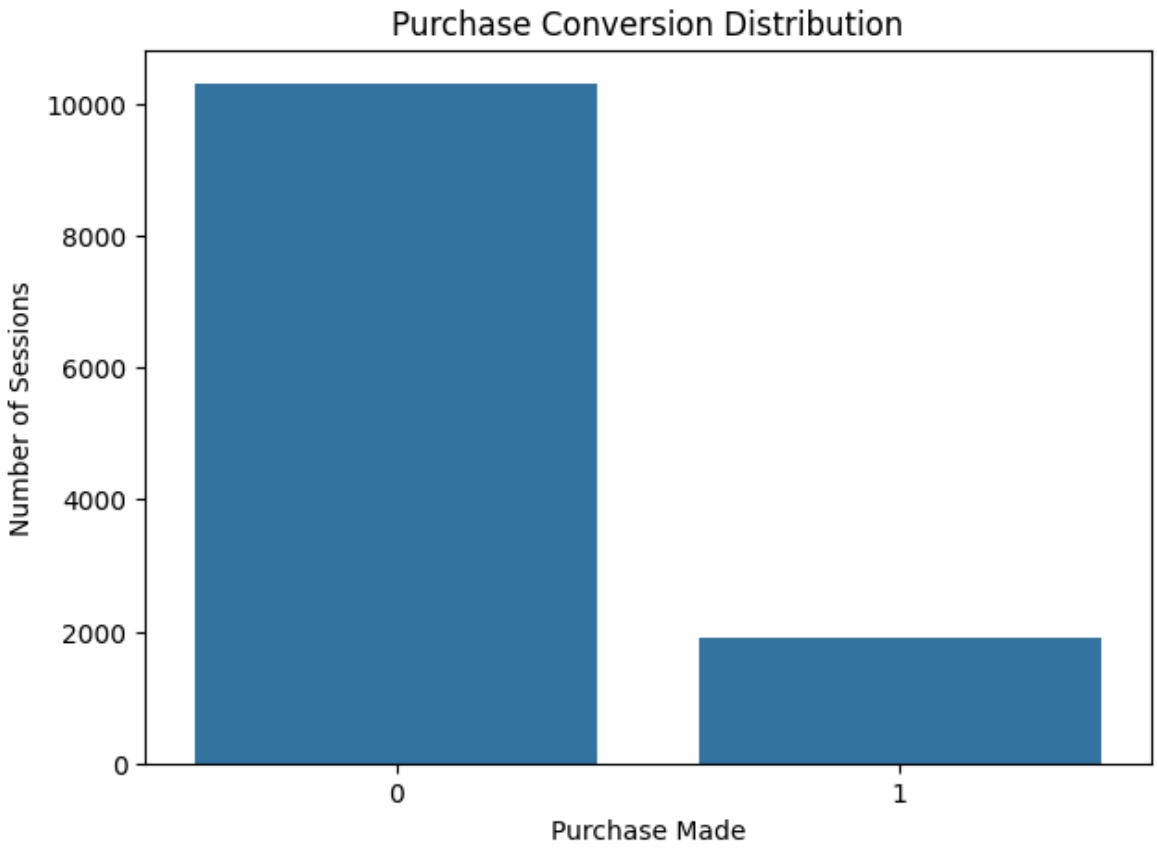
In [14]:
```
df["Revenue"] = df["Revenue"].astype(int)
```

In [15]:
```
plt.figure(figsize=(7,5))

sns.countplot(data=df, x="Revenue")

plt.title("Purchase Conversion Distribution")
plt.xlabel("Purchase Made")
plt.ylabel("Number of Sessions")

plt.show()
```



In [ ]:
```
traffic type
```

In [16]:
```
traffic_conversion = (
    df.groupby("TrafficType")["Revenue"]
    .mean()
    .sort_values(ascending=False)
)

print(traffic_conversion)
```

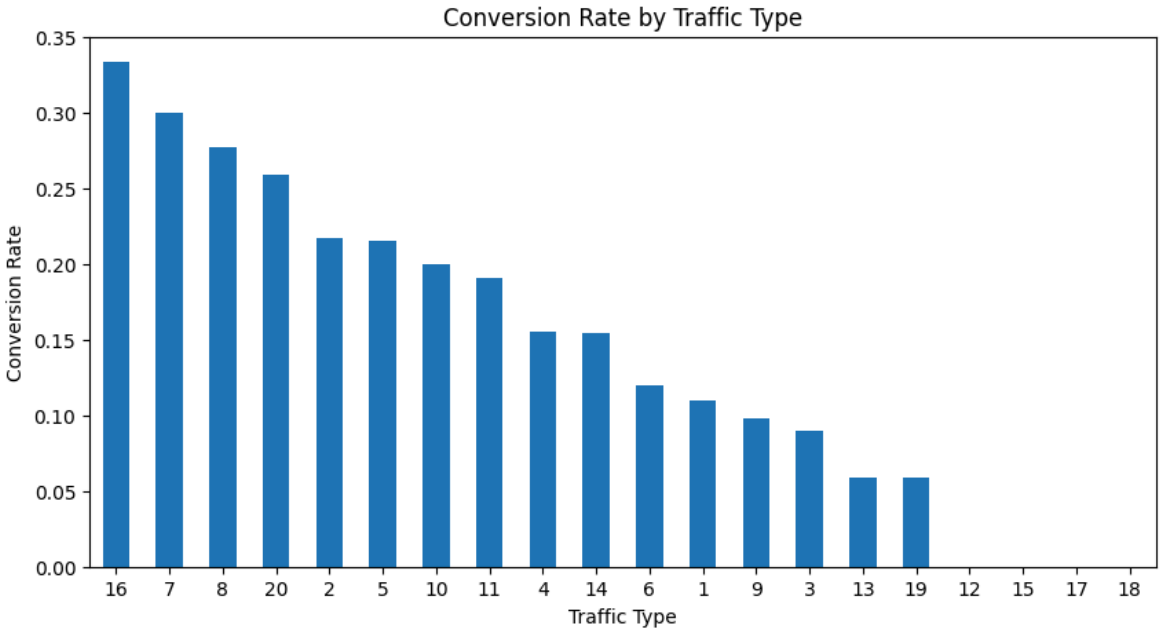
```
TrafficType
16    0.333333
7     0.300000
8     0.276968
20    0.259067
2     0.216569
5     0.215385
10    0.200000
11    0.190283
4     0.154784
14    0.153846
6     0.119639
1     0.109715
9     0.097561
3     0.089419
13    0.059066
19    0.058824
12    0.000000
15    0.000000
17    0.000000
18    0.000000
Name: Revenue, dtype: float64
```

In [17]:
```
plt.figure(figsize=(10,5))

traffic_conversion.plot(kind="bar")

plt.title("Conversion Rate by Traffic Type")
plt.xlabel("Traffic Type")
plt.ylabel("Conversion Rate")

plt.xticks(rotation=0)
plt.show()
```



In [18]:
```
visitor_conversion = (
    df.groupby("VisitorType")["Revenue"]
    .mean()
    .sort_values(ascending=False)
)

print(visitor_conversion)
```

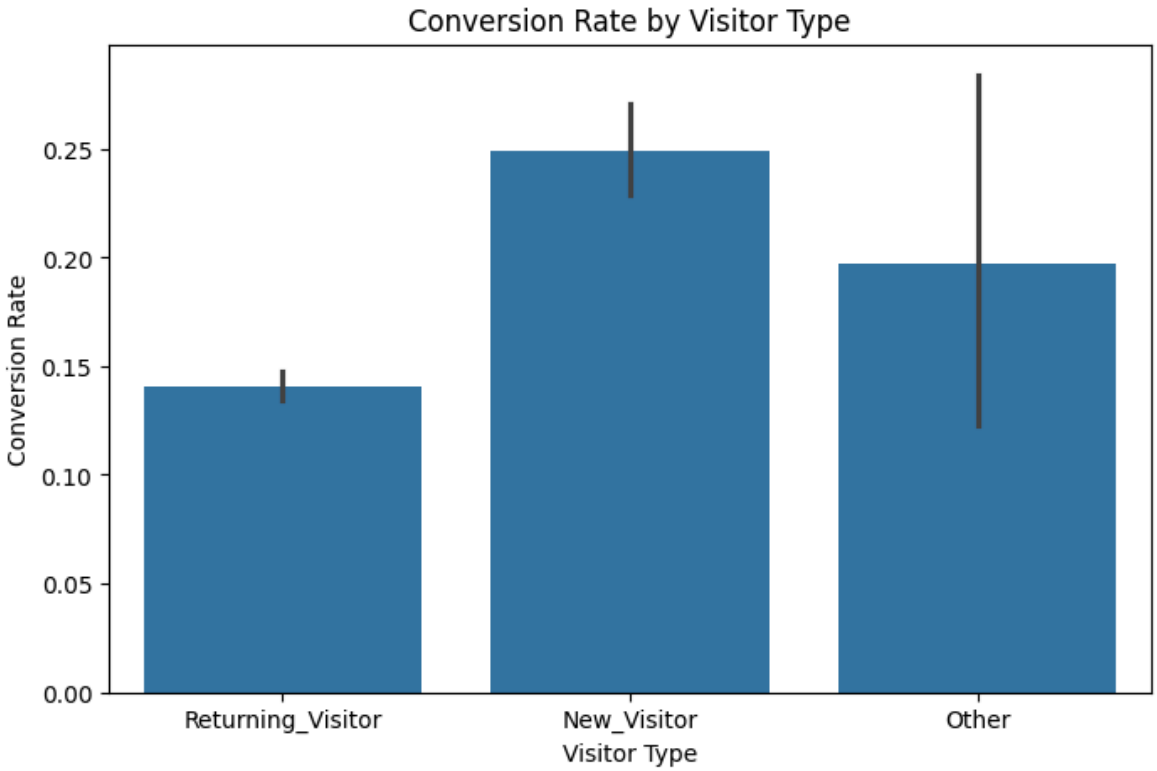
```
VisitorType
New_Visitor          0.249262
Other                0.197531
Returning_Visitor    0.140926
Name: Revenue, dtype: float64
```

In [19]:
```
plt.figure(figsize=(8,5))

sns.barplot(
    data=df,
    x="VisitorType",
    y="Revenue"
)

plt.title("Conversion Rate by Visitor Type")
plt.xlabel("Visitor Type")
plt.ylabel("Conversion Rate")

plt.show()
```



In [20]:
```
df["Total_Duration"] = (
    df["Administrative_Duration"]
    + df["Informational_Duration"]
    + df["ProductRelated_Duration"]
)

df[[
    "Administrative_Duration",
    "Informational_Duration",
    "ProductRelated_Duration",
    "Total_Duration"
]].head()
```

Out[20]:

	Administrative_Duration	Informational_Duration	ProductRelated_Duration	Total_Dura
0	0.0	0.0	0.000000	0.00
1	0.0	0.0	64.000000	64.00
2	0.0	0.0	0.000000	0.00
3	0.0	0.0	2.666667	2.66
4	0.0	0.0	627.500000	627.50

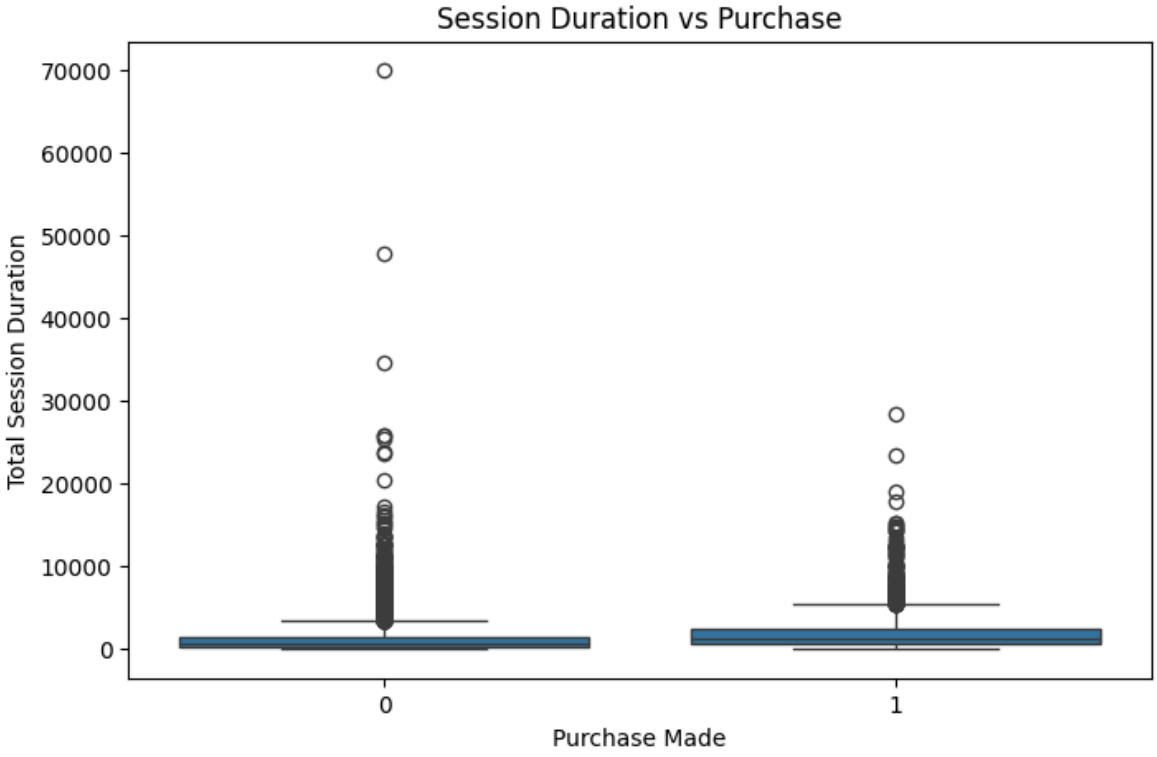
In [21]:
```
plt.figure(figsize=(8,5))

sns.boxplot(
    data=df,
    x="Revenue",
    y="Total_Duration"
)

plt.title("Session Duration vs Purchase")
```

```
plt.xlabel("Purchase Made")
plt.ylabel("Total Session Duration")

plt.show()
```

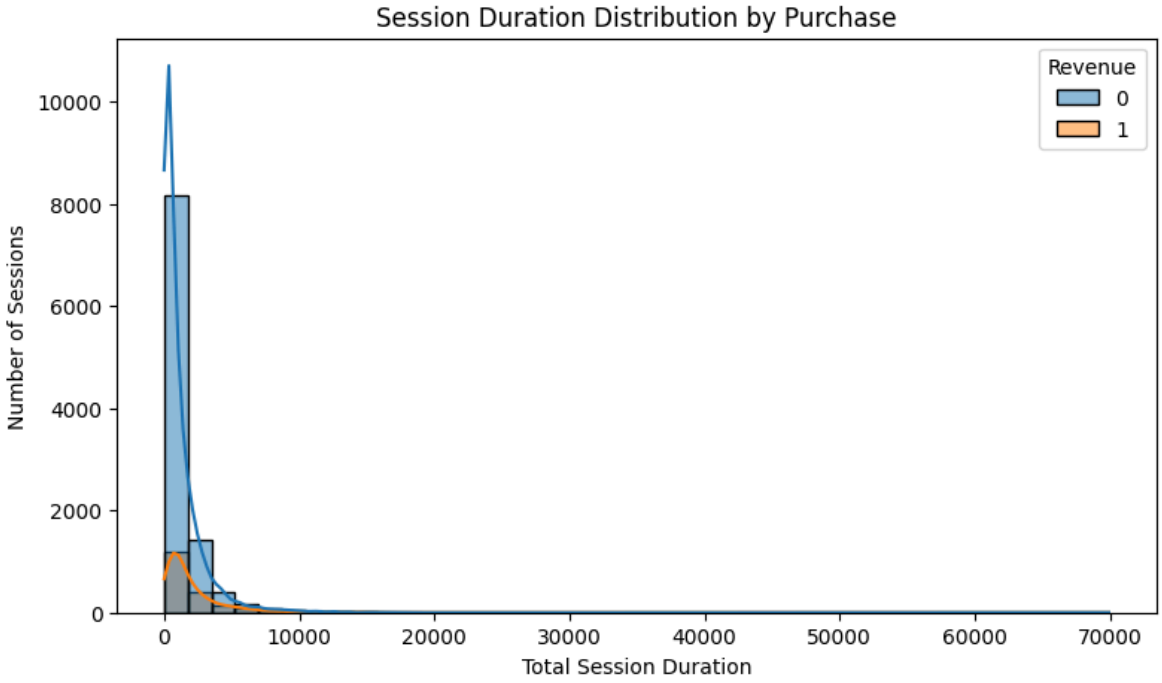


In [22]:
```
plt.figure(figsize=(9,5))

sns.histplot(
    data=df,
    x="Total_Duration",
    hue="Revenue",
    kde=True,
    bins=40
)

plt.title("Session Duration Distribution by Purchase")
plt.xlabel("Total Session Duration")
plt.ylabel("Number of Sessions")

plt.show()
```



In [23]:
```
df["Total_Pages"] = (
    df["Administrative"]
    + df["Informational"]
    + df["ProductRelated"]
)

df[[
    "Administrative",
    "Informational",
    "ProductRelated",
    "Total_Pages"
]].head()
```

Out[23]:

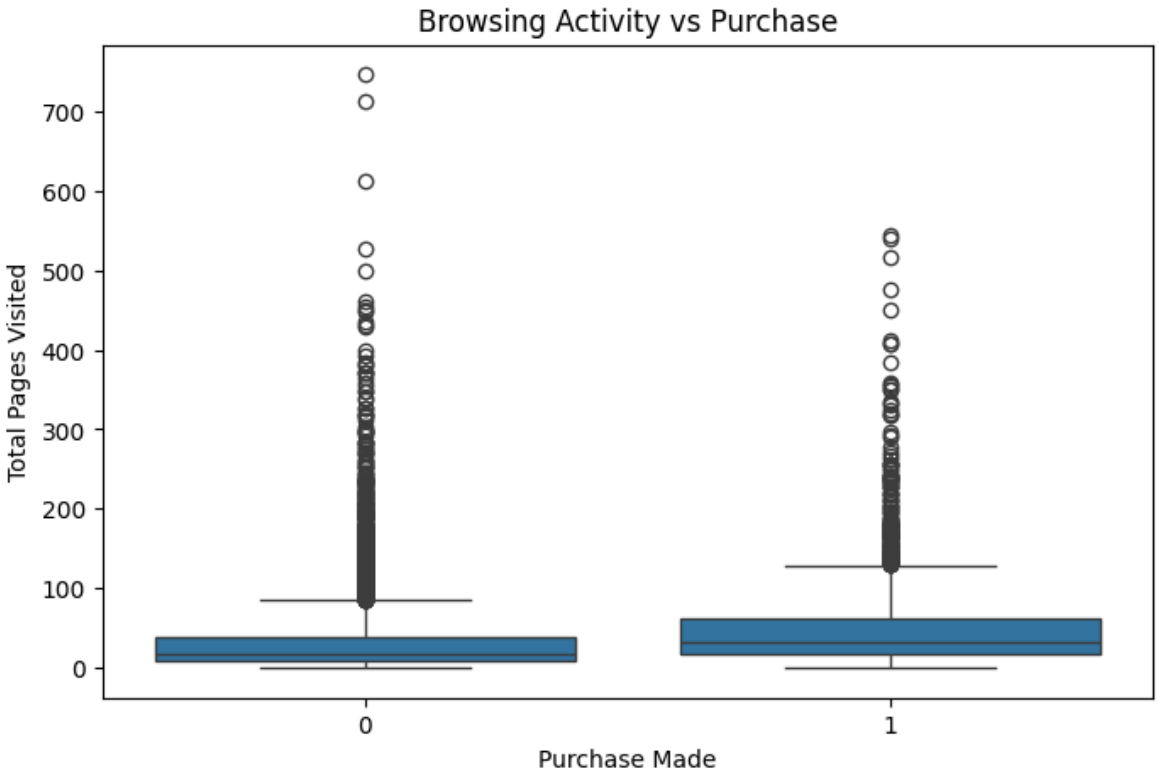
	Administrative	Informational	ProductRelated	Total_Pages
0	0	0	1	1
1	0	0	2	2
2	0	0	1	1
3	0	0	2	2
4	0	0	10	10

In [24]:
```
plt.figure(figsize=(8,5))

sns.boxplot(
    data=df,
    x="Revenue",
    y="Total_Pages"
)

plt.title("Browsing Activity vs Purchase")
plt.xlabel("Purchase Made")
plt.ylabel("Total Pages Visited")

plt.show()
```

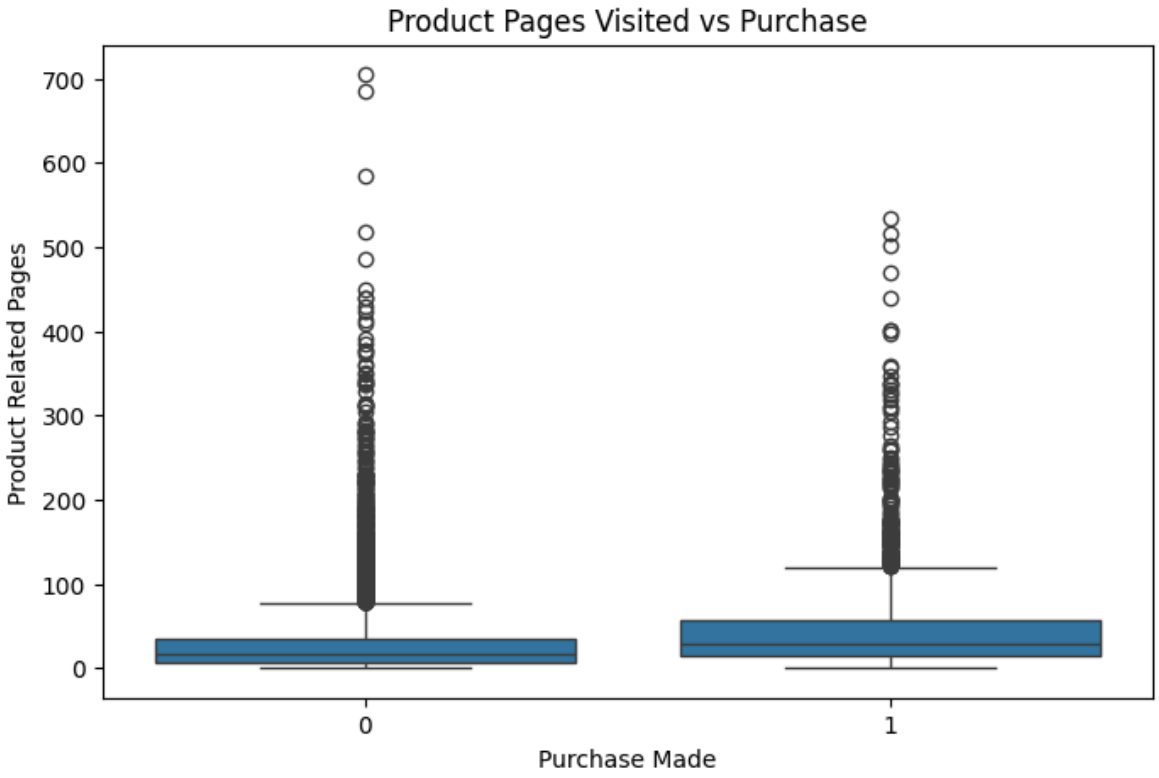


In [25]:
```
plt.figure(figsize=(8,5))

sns.boxplot(
    data=df,
    x="Revenue",
    y="ProductRelated"
)

plt.title("Product Pages Visited vs Purchase")
plt.xlabel("Purchase Made")
plt.ylabel("Product Related Pages")

plt.show()
```



In [26]:
```
device_conversion = (
    df.groupby("OperatingSystems")["Revenue"]
    .mean()
    .sort_values(ascending=False)
)

print(device_conversion)
```

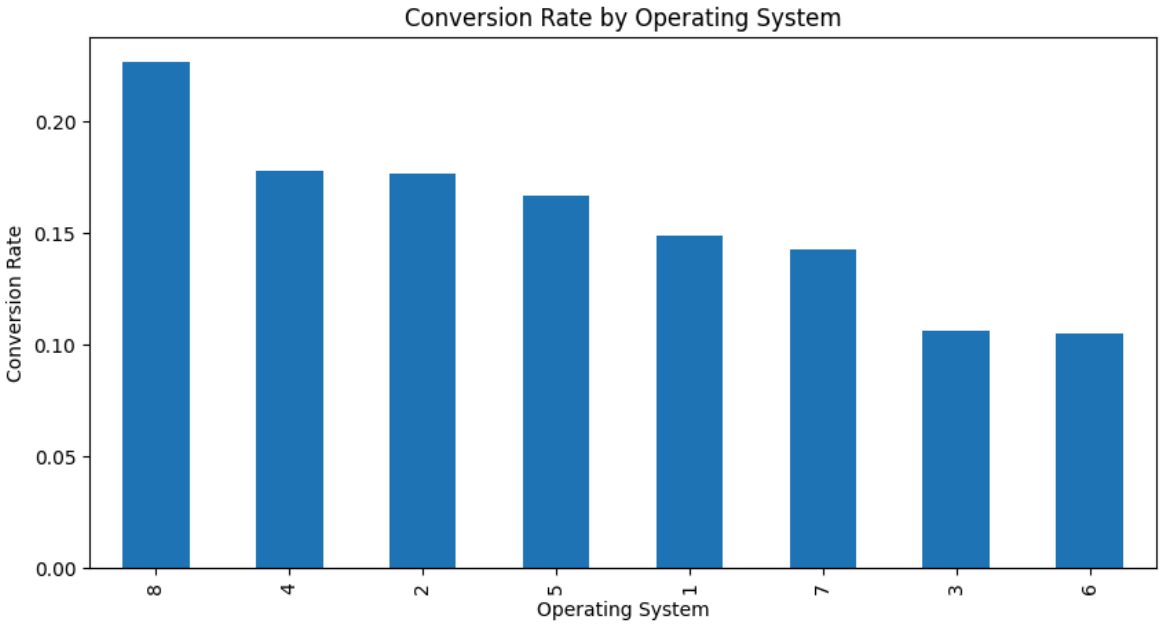
```
OperatingSystems
8    0.226667
4    0.177824
2    0.176579
5    0.166667
1    0.148686
7    0.142857
3    0.105929
6    0.105263
Name: Revenue, dtype: float64
```

In [27]:
```
plt.figure(figsize=(10,5))

device_conversion.plot(kind="bar")

plt.title("Conversion Rate by Operating System")
plt.xlabel("Operating System")
plt.ylabel("Conversion Rate")

plt.show()
```

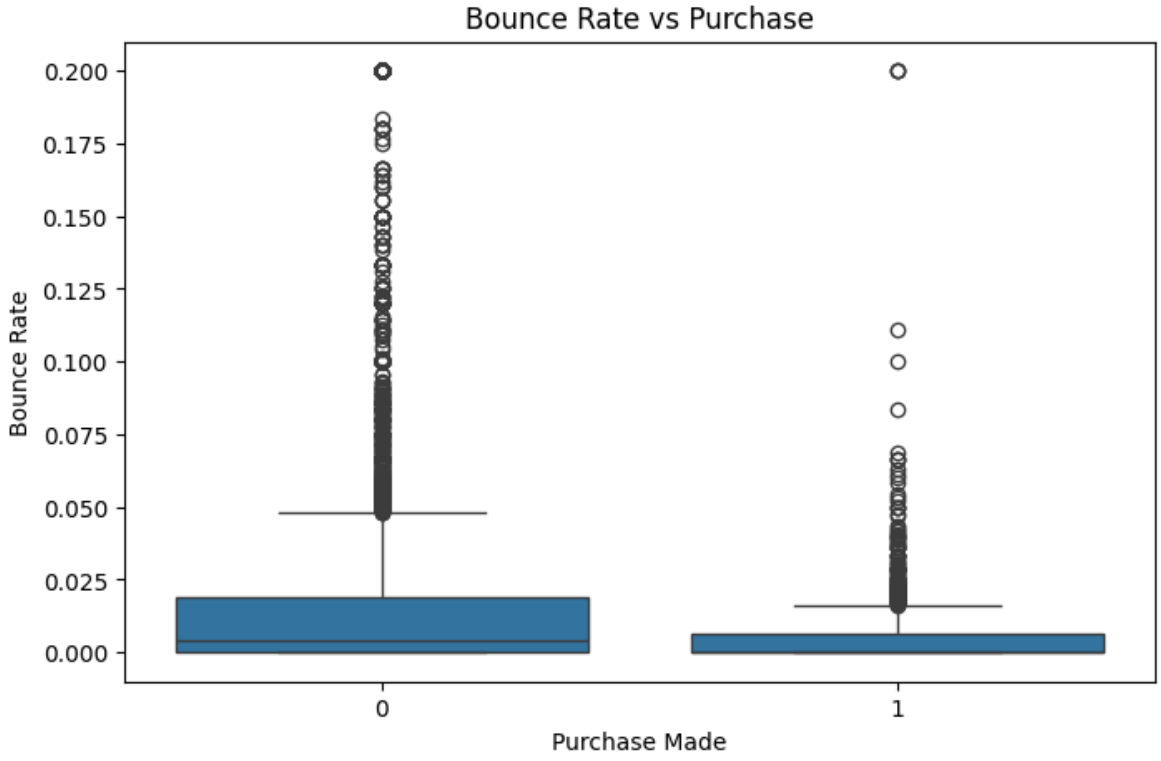


In [28]:
```
plt.figure(figsize=(8,5))

sns.boxplot(
    data=df,
    x="Revenue",
    y="BounceRates"
)

plt.title("Bounce Rate vs Purchase")
plt.xlabel("Purchase Made")
plt.ylabel("Bounce Rate")

plt.show()
```

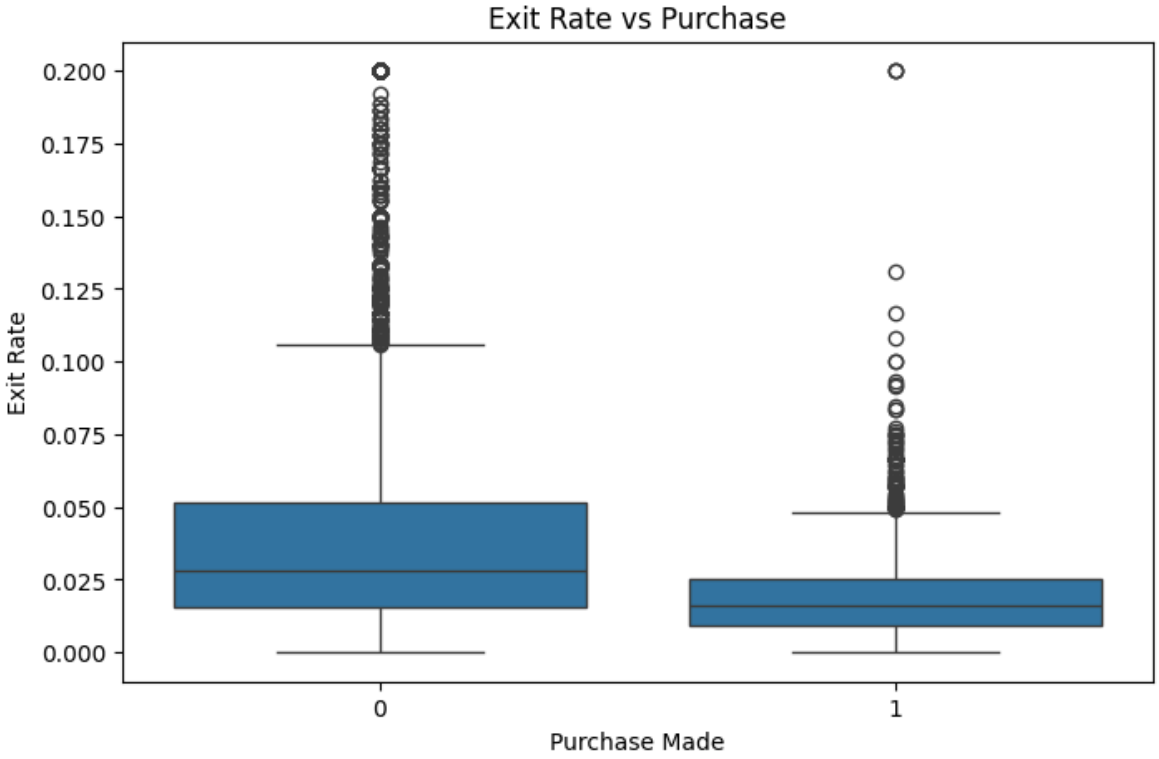


In [29]:
```
plt.figure(figsize=(8,5))
```

```
sns.boxplot(
    data=df,
    x="Revenue",
    y="ExitRates"
)

plt.title("Exit Rate vs Purchase")
plt.xlabel("Purchase Made")
plt.ylabel("Exit Rate")

plt.show()
```

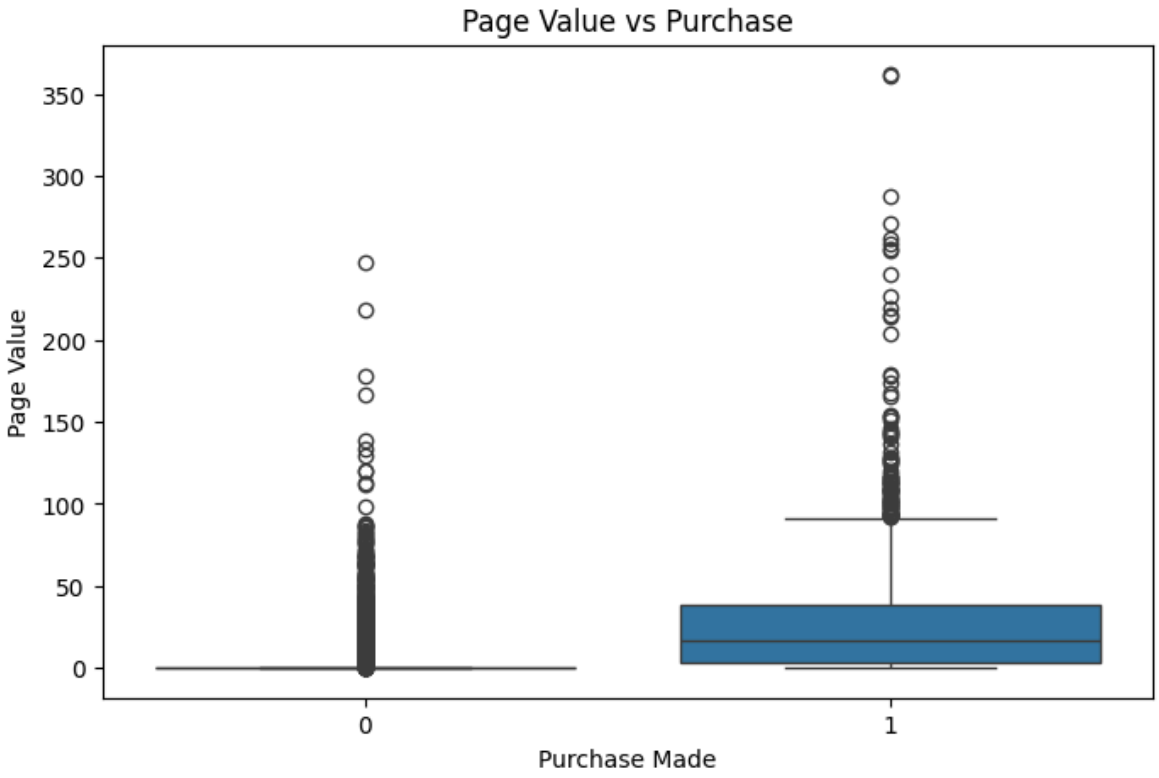


In [30]:
```
plt.figure(figsize=(8,5))

sns.boxplot(
    data=df,
    x="Revenue",
    y="PageValues"
)

plt.title("Page Value vs Purchase")
plt.xlabel("Purchase Made")
plt.ylabel("Page Value")

plt.show()
```



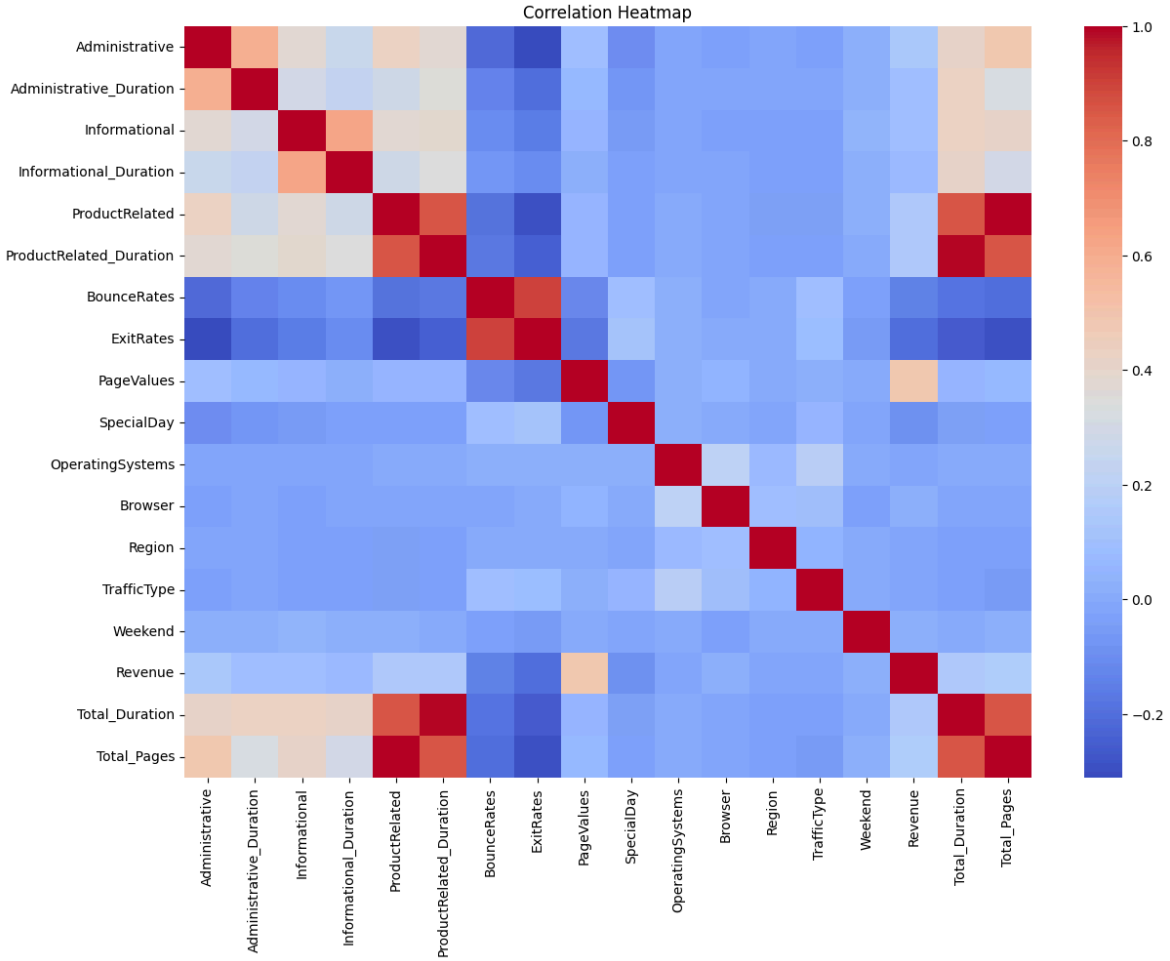
In [31]:
```
numeric_df = df.select_dtypes(include=["int64", "float64", "bool"])

plt.figure(figsize=(14,10))

sns.heatmap(
    numeric_df.corr(),
    cmap="coolwarm",
    annot=False
)

plt.title("Correlation Heatmap")

plt.show()
```



In [32]:
```
X = df.drop("Revenue", axis=1)
y = df["Revenue"]
```

In [33]:
```
categorical_cols = X.select_dtypes(
    include=["object", "bool"]
).columns.tolist()

numerical_cols = X.select_dtypes(
    include=["int64", "float64"]
).columns.tolist()

print("Categorical columns:")
print(categorical_cols)

print("\nNumerical columns:")
print(numerical_cols)
```

```
Categorical columns:
['Month', 'VisitorType', 'Weekend']

Numerical columns:
['Administrative', 'Administrative_Duration', 'Informational', 'Informational_Dur
ation', 'ProductRelated', 'ProductRelated_Duration', 'BounceRates', 'ExitRates',
'PageValues', 'SpecialDay', 'OperatingSystems', 'Browser', 'Region', 'TrafficTyp
e', 'Total_Duration', 'Total_Pages']
```

In [34]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
```

```
    random_state=42,
    stratify=y
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (9764, 19)
Testing data: (2441, 19)
```

In [35]:
```
preprocessor = ColumnTransformer(
    transformers=[
        (
            "num",
            StandardScaler(),
            numerical_cols
        ),
        (
            "cat",
            OneHotEncoder(
                handle_unknown="ignore",
                sparse_output=False
            ),
            categorical_cols
        )
    ]
)
```

In [36]:
```
decision_tree = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        (
            "classifier",
            DecisionTreeClassifier(
                random_state=42,
                max_depth=6
            )
        )
    ]
)

decision_tree.fit(X_train, y_train)

y_pred_dt = decision_tree.predict(X_test)
```

In [37]:
```
accuracy_dt = accuracy_score(y_test, y_pred_dt)
precision_dt = precision_score(y_test, y_pred_dt)
recall_dt = recall_score(y_test, y_pred_dt)
f1_dt = f1_score(y_test, y_pred_dt)

print("Decision Tree Results")
print("---------------------")
print("Accuracy :", accuracy_dt)
print("Precision:", precision_dt)
print("Recall   :", recall_dt)
print("F1 Score :", f1_dt)
```

```
Decision Tree Results
--------------------
Accuracy : 0.8975829578041786
Precision: 0.6941176470588235
Recall   : 0.6178010471204188
F1 Score : 0.6537396121883656
```

In [38]:
```
print(classification_report(y_test, y_pred_dt))
```

```
              precision    recall  f1-score   support

           0       0.93      0.95      0.94      2059
           1       0.69      0.62      0.65       382

    accuracy                           0.90      2441
   macro avg       0.81      0.78      0.80      2441
weighted avg       0.89      0.90      0.90      2441
```

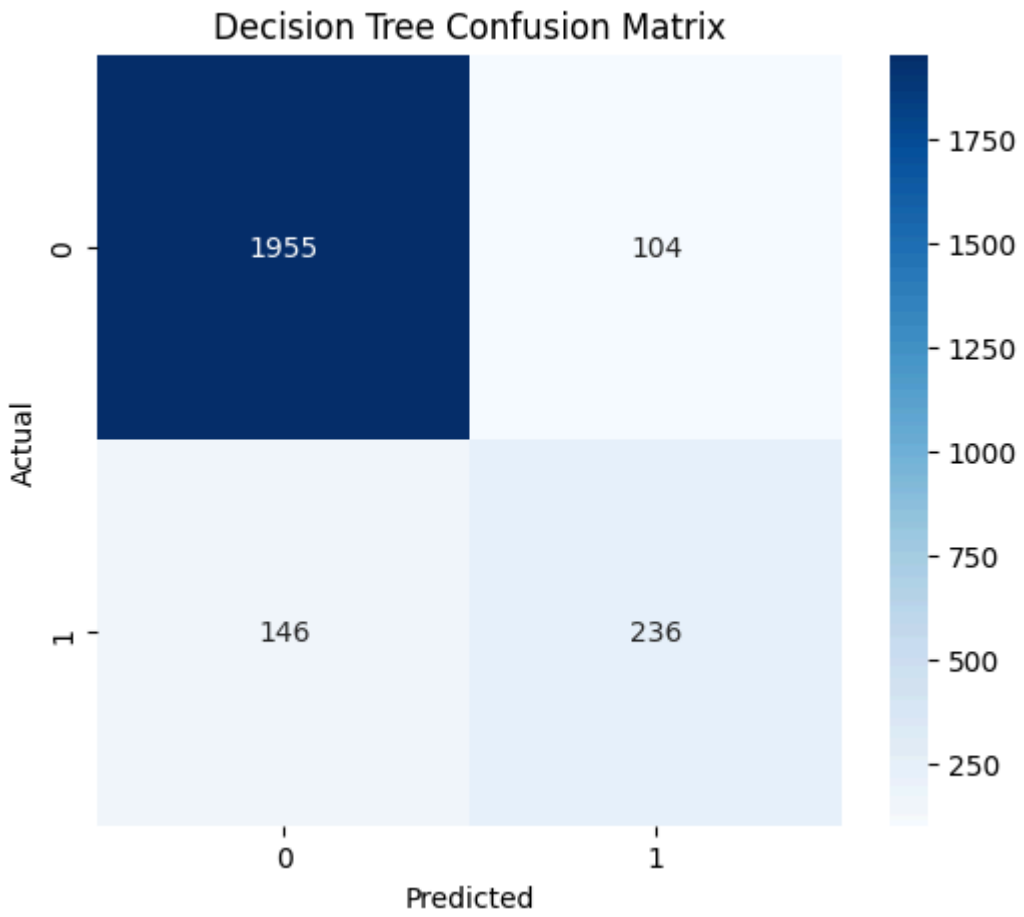
In [39]:
```
cm_dt = confusion_matrix(y_test, y_pred_dt)

plt.figure(figsize=(6,5))

sns.heatmap(
    cm_dt,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.title("Decision Tree Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```



In [ ]:
```
True Negative
False Positive
False Negative
True Positive
```

In [40]:
```
svm_model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        (
            "classifier",
            SVC(
                kernel="rbf",
                random_state=42
            )
        )
    ]
)

svm_model.fit(X_train, y_train)

y_pred_svm = svm_model.predict(X_test)
```

In [41]:
```
accuracy_svm = accuracy_score(y_test, y_pred_svm)
precision_svm = precision_score(y_test, y_pred_svm)
recall_svm = recall_score(y_test, y_pred_svm)
f1_svm = f1_score(y_test, y_pred_svm)

print("SVM Results")
print("-----------")
print("Accuracy :", accuracy_svm)
print("Precision:", precision_svm)
```

```
print("Recall   :", recall_svm)
print("F1 Score :", f1_svm)
```

```
SVM Results
-----------
Accuracy : 0.8943056124539124
Precision: 0.7403100775193798
Recall   : 0.5
F1 Score : 0.596875
```

In [42]:
```
print(classification_report(y_test, y_pred_svm))
```

```
              precision    recall  f1-score   support

           0       0.91      0.97      0.94      2059
           1       0.74      0.50      0.60       382

    accuracy                           0.89      2441
   macro avg       0.83      0.73      0.77      2441
weighted avg       0.89      0.89      0.89      2441
```

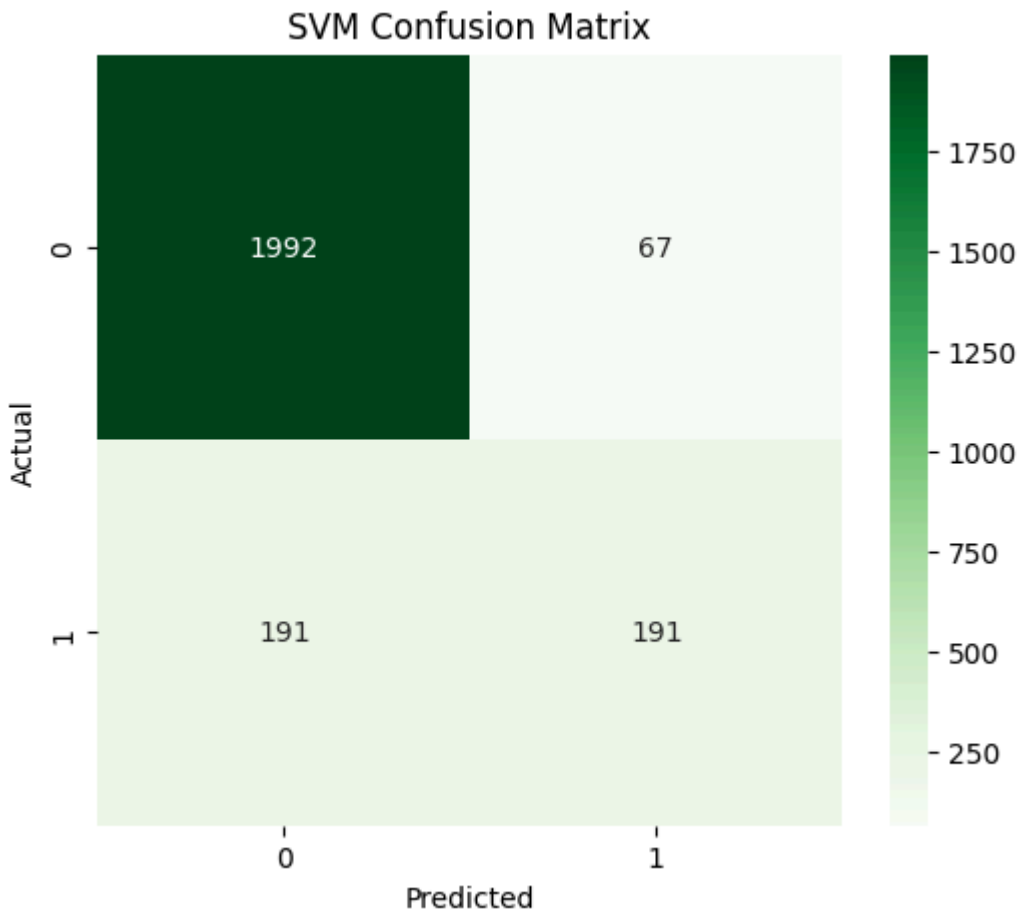
In [43]:
```
cm_svm = confusion_matrix(y_test, y_pred_svm)

plt.figure(figsize=(6,5))

sns.heatmap(
    cm_svm,
    annot=True,
    fmt="d",
    cmap="Greens"
)

plt.title("SVM Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```



In [44]:
```
results = pd.DataFrame({
    "Model": ["Decision Tree", "SVM"],
    "Accuracy": [accuracy_dt, accuracy_svm],
    "Precision": [precision_dt, precision_svm],
    "Recall": [recall_dt, recall_svm],
    "F1 Score": [f1_dt, f1_svm]
})

results
```

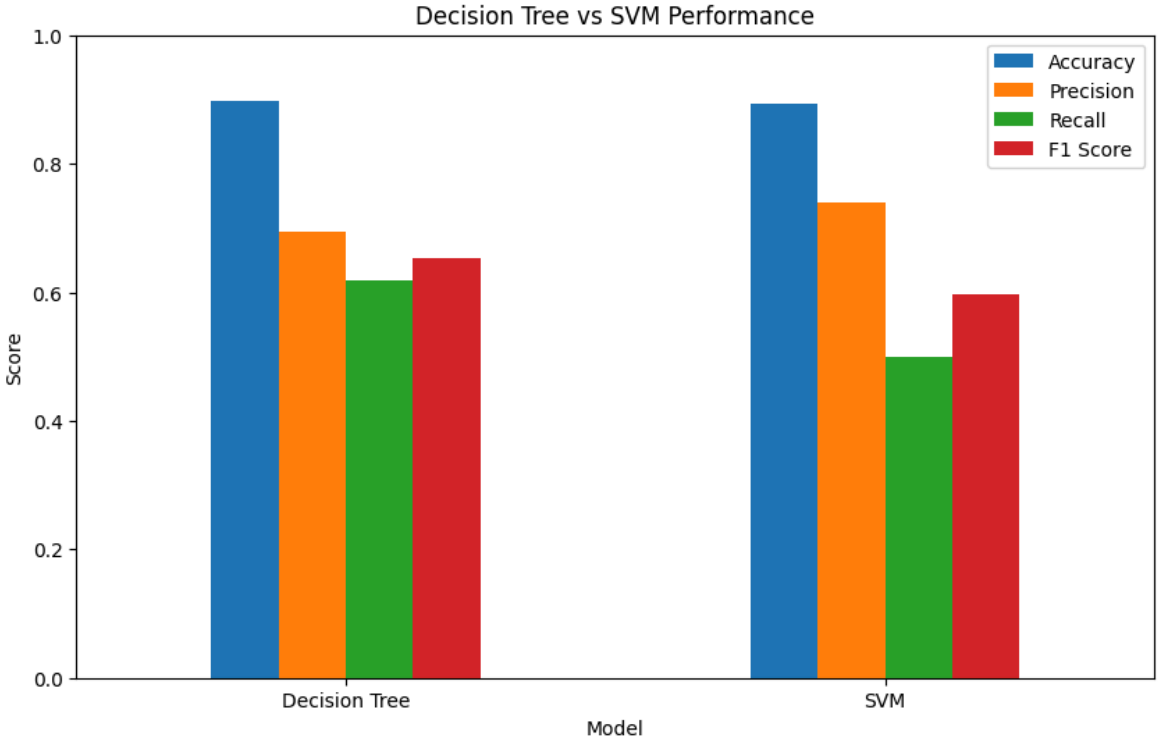
Out[44]:

	Model	Accuracy	Precision	Recall	F1 Score
0	Decision Tree	0.897583	0.694118	0.617801	0.653740
1	SVM	0.894306	0.740310	0.500000	0.596875

In [45]:
```
results.set_index("Model").plot(
    kind="bar",
    figsize=(10,6)
)

plt.title("Decision Tree vs SVM Performance")
plt.ylabel("Score")
plt.ylim(0,1)

plt.xticks(rotation=0)
plt.show()
```



In [46]:
```
dt_preprocessor = decision_tree.named_steps["preprocessor"]
dt_model = decision_tree.named_steps["classifier"]

feature_names = dt_preprocessor.get_feature_names_out()

importance = dt_model.feature_importances_

feature_importance = pd.DataFrame({
    "Feature": feature_names,
    "Importance": importance
})

feature_importance = feature_importance.sort_values(
    by="Importance",
    ascending=False
)

feature_importance.head(15)
```

Out[46]:

	Feature	Importance
8	num__PageValues	0.736767
6	num__BounceRates	0.072009
23	cat__Month_Nov	0.043509
14	num__Total_Duration	0.028352
5	num__ProductRelated_Duration	0.027931
0	num__Administrative	0.021822
1	num__Administrative_Duration	0.021799
7	num__ExitRates	0.020291
15	num__Total_Pages	0.008884
21	cat__Month_Mar	0.005156
10	num__OperatingSystems	0.004097
25	cat__Month_Sep	0.003138
30	cat__Weekend_True	0.002164
19	cat__Month_Jul	0.001263
17	cat__Month_Dec	0.000953

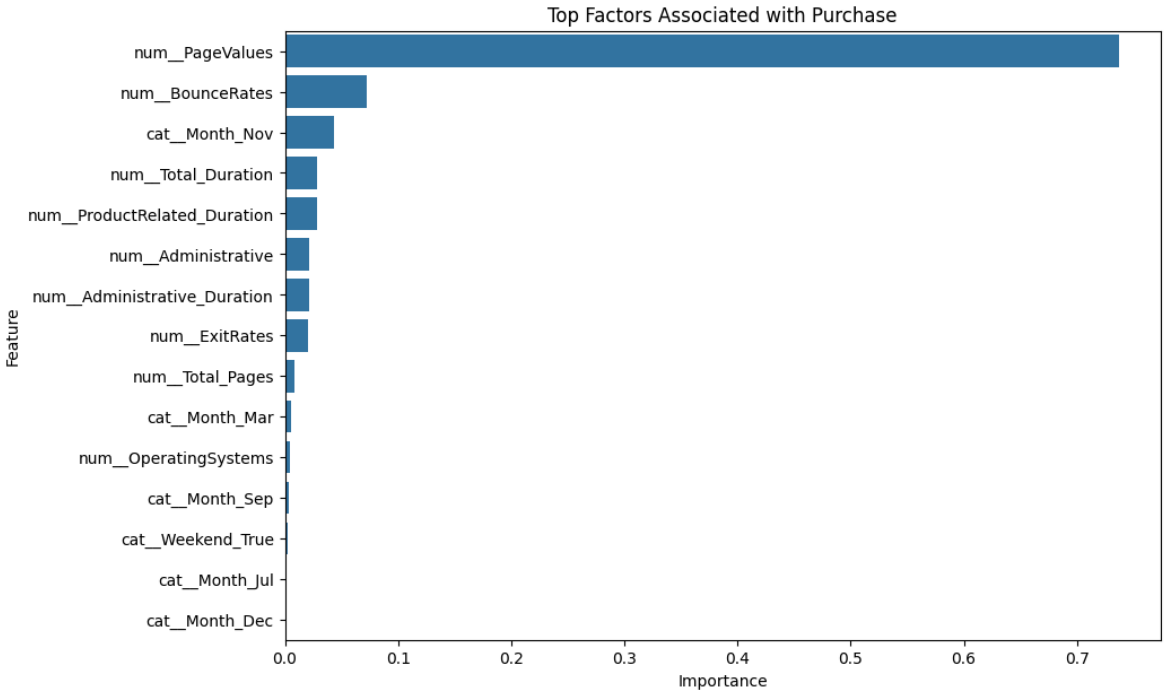
In [47]:
```
top_features = feature_importance.head(15)

plt.figure(figsize=(10,7))

sns.barplot(
    data=top_features,
    x="Importance",
    y="Feature"
)

plt.title("Top Factors Associated with Purchase")
plt.xlabel("Importance")
plt.ylabel("Feature")

plt.show()
```


Top Factors Associated with Purchase
num__PageValues
num__BounceRates
cat__Month_Nov
num__Total_Duration
num__ProductRelated_Duration
num__Administrative
num__Administrative_Duration
num__ExitRates
num__Total_Pages
cat__Month_Mar
num__OperatingSystems
cat__Month_Sep
cat__Weekend_True
cat__Month_Jul
cat__Month_Dec
Feature
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
Importance

In [ ]: